Álgebra de *Boole*

Introdução

Em Matemática, chama-se ***proposição*** ao enunciado de uma verdade que se quer demonstrar, ou como usaremos: uma sentença que pode ser falsa (0), ou verdadeira (1), mas nunca ambos ao mesmo tempo.

- A **conjunção** é uma relação entre sentenças que estabelece um resultado **verdadeiro** (1) quando associadas duas proposições (p e q), ambas **verdadeiras** (iguais a 1). Basta uma delas ser **falsa** (0), para que a conjunção (s) também seja **falsa** (0).
  - A porta **AND** (E) é um componente de circuito lógico que implementa essa relação; pode ter duas (p, q), ou mais entradas, e a saída (s) assumirá o valor 1 (**verdadeiro**) se, e somente se, todas as entradas forem iguais a 1 (**verdadeiras**); caso uma, ou mais entradas forem iguais a 0 (**falso**), a saída terá valor igual a 0 (**falso**).
- A **disjunção** é uma relação entre sentenças que estabelece um resultado **falso** (0) quando duas proposições (p e q) forem **falsas** (0). Basta uma delas ser **verdadeira** (1), para que a disjunção também seja **verdadeira** (1).
  - A porta **OR** (OU) é um componente de circuito lógico que implementa essa relação; pode ter duas (p, q), ou mais entradas, e a saída (s) assumirá o valor 0 (**falso**) se, e somente se, todas as entradas forem iguais a 0 (**falso**); caso uma, ou mais entradas forem iguais a 1 (**verdadeiro**), a saída terá valor 1 (**verdadeiro**).
- A **negação** determina que se uma proposição (p) for **falsa** (0), seu resultado será **verdadeiro** (1), ou vice-versa.
  - A porta **NOT** (NÃO) é um componente de circuito lógico que implementa essa relação, também chamada de **INVERTER** (INVERSOR), só possui uma entrada (p), e a saída assumirá o valor 1 (**verdadeiro**), se a entrada for igual a 0 (**falso**); senão, a saída terá valor 0 (**falso**), se a entrada for igual a 1 (**verdadeiro**).

Representações de Relações Lógicas

- Notações :

```
Conjunção ( E )     Disjunção ( OU )      Negação ( NÃO )
   (p e q)             (p ou q)              (não p)
    p ∧ q               p ∨ q                 ¬p
p . q  =  p q           p + q               /p = p' = p
    p & q               p | q                 ~p
    p && q              p || q               ! p
```

- Tabela-verdade:

```
Conjunção ( E )     Disjunção ( OU )     Negação (NÃO )
  p   q      s        p   q     s          p      s
  0 . 0   =  0        0 + 0  =  0
  0 . 1   =  0        0 + 1  =  1          0'  =  1
  1 . 0   =  0        1 + 0  =  1          1'  =  0
  1 . 1   =  1        1 + 1  =  1
```

- Portas-lógicas:

```
s = AND ( p, q )     s = OR ( p, q )     s = NOT( p )
     _____                _____               _____
p --|     |          p --|     |             |     |\
    |  &  |-- s          | ≥1  |-- s     p --|  !  |-- s
q --|_____|          q --|_____|             |_____|
```

Prioridade de Conectivos

Na álgebra de *Boole* estabelece-se que a ordem de avaliação de uma expressão, envolvendo conectivos lógicos, será da **esquerda para a direita**. Entretanto, para certas aplicações (linguagens de programação, por exemplo) é usual atribuir prioridades aos conectivos como mostrado abaixo, sendo a primeira a mais alta, quando aplicada imediatamente a um valor.

```
NÃO
E
OU
```

Pode-se alterar a ordem de avaliação por meio de parênteses.

- Exemplo:

```
Considere a expressão lógica: ( !x && y )  || ( x && !y )
De forma mais simples como: ( x' . y ) + ( x . y' ).

A sua avaliação será feita na seguinte ordem de prioridade:
  - negação de (x)    : x'
  - conjunção com (y) : x'. y
  - negação de (y)    : y'
  - conjunção com (x) : x . y'
  - disjunção das conjunções : (x' . y) + (x . y')

A expressão poderá ser representada nas formas tabular ( tabela-verdade )
ou por BDD ( Binary Decision Diagram ).

  | n | x y | (x' . y) | (x . y') | (x'.y)+(x.y') |
```

```
|---|-----|----------|----------|---------------|
| 0 | 0 0 |    0     |    0     |       0       |
| 1 | 0 1 |    1     |    0     |       1       |
| 2 | 1 0 |    0     |    1     |       1       |
| 3 | 1 1 |    0     |    0     |       0       |
```

Resumidamente as relações em uma tabela também poderão ser indicadas :

- Pela disjunção (+) das conjunções iguais a 1 (ou mintermos) :

```
| n |    m   | x y | (x'. y) | (x . y') | (x'.y)+(x.y') |  mintermos ( = 1
)
|---|------- |-----|---------|----------|---------------|
| 0 | x'. y' | 0 0 |    0    |    0     |       0       |  m0 = x'. y' = 0
| 1 | x'. y  | 0 1 |    1    |    0     |       1       |  m1 = x'. y  = 1
←
| 2 | x . y' | 1 0 |    0    |    1     |       1       |  m2 = x . y' = 2
←
| 3 | x . y  | 1 1 |    0    |    0     |       0       |  m3 = x . y  = 3
```

$x \quad x. \quad x. \quad 1\ 2 \quad 1\ 2 \quad 1\ 2$

- Pela conjunção (.) das disjunções iguais a 0 (ou MAXTERMOS).

```
| N |   M   | X Y | (X+Y') | (X'+Y) | (X+Y').(X'+Y) |  MAXTERMOS ( = 0 )
|---|-------|-----|--------|--------|---------------|
| 0 | X +Y  | 0 0 |   0    |    0   |       0       |  M0 = X  + Y  = 0 ←
| 1 | X +Y' | 0 1 |   1    |    0   |       1       |  M1 = X  + Y' = 1
| 2 | X'+Y  | 1 0 |   0    |    1   |       1       |  M2 = X' + Y  = 2
| 3 | X'+Y' | 1 1 |   0    |    0   |       0       |  M3 = X' + Y' = 3 ←
```

$. \quad 0.3 \quad 0\ 3 \quad 0\ 3$

Principais Relações

mintermos	MAXTERMOS	x	y	AND	OR	XOR	XNOR	NAND	NOR
m	M	p	q	p & q	p | q	p ^ q	~(p ^ q)	~(p & q)	~(p | q)
(p' . q')	(P + Q)	0	0	0	0	0	1	1	1
(p' . q)	(P + Q')	0	1	0	1	1	0	1	0
(p . q')	(P' + Q)	1	0	0	1	1	0	1	0
(p . q)	(P' + Q')	1	1	1	1	0	1	0	0

```
                          +-------+-------+-----+-----+-------+-------+
mintermos  SoP (+) [=1] | 3     | 1,2,3 | 1,2 | 0,3 | 0     | 0,1,2 |
                          |-------|-------|-----|-----|-------|-------|
MAXTERMOS  PoS (.) [=0] | 0,1,2 | 0     | 0,3 | 1,2 | 1,2,3 | 3     |
                          +-------+-------+-----+-----+-------+-------+
```

Principais Propriedades

Idempotência	Comutativa	Associativa
p + p = p	p + q = q + p	(p + q) + r = p + (q + r)
p . p = p	p . q = q . p	(p . q) . r = p . (q . r)

Identidade	De Morgan	Distributiva
p + 0 = p p . 0 = 0	(p + q)' = p' . q'	p + (q . r) = (p + q) . (p + r)
p + 1 = 1 p . 1 = p	(p . q)' = p' + q'	p . (q + r) = (p . q) + (p . r)

Complementar	Absorção	Consenso
p + p' = 1 (tautologia)	p + (p' . q) = p + q	(p . q) + (p'. r) + (q . r) = (p . q) + (p' . r)
p . p' = 0 (contradição)	p' + (p . q) = p' + q	(p + q) . (p + r) . (q + r) = (p + q) . (p + r)
p'' = p (dupla negação)	p + (p . q) = p	

Principais Propriedades com XOR

Básicas	Identidade	Complementar
p ^ p = 0	p ^ 0 = p	p ^ q = p ^ q
p ^ p = 1	p ^ 1 = p	(p ^ q)' = p' ^ q = p ^ q'

Associativa	Comutativa
(p ^ q) ^ r = p ^ (q ^ r)	p ^ q = q ^ p

Disjunção	Distributiva	Transposição
se: (p = q ^ r) e (q . r = 0), então: (p = q + r)	p . (q ^ r) = (p . q) ^ (p . r)	se: (p = q ^ r), então: (q = p ^ r) e (r = p ^ q)

Universalidade das portas NAND e NOR

- Negação

```
(a.b)' => (a.a)'   = a'+a' = a'

nand(a,a) = not(a) = a'

(a+b)' => (a+a)'   = a'.a' = a'

nor (a,a) = not(a) = a'
```

- Substituição

```
(a'.b')' => a+b

nand(not(a), not(b)) = p = or (a,b)

(a'+b')' => a.b

nor (not(a), not(b)) = nor (nor (a,a), nor (b,b)) = and(a,b)
```

- Identidade

```
nand( nand(a,a), nand(a,a) ) = nand( not(a), not(a) ) = not( not(a) ) = a
nand( nand(a,a), nand(a,a) ) = or(a.a)                                 = a

nor( nor(a,a), nor (a,a) ) = nor(not(a), not(a)) = not(not(a)) = a
nor( nor(a,a), nor (a,a) ) = and(a,a)                          = a

nand( nand(p,q) )           = and(p,q)
nor( nor(p,p), nor(q,q) )   = and(p,q)
nor( not(p), not(q) )       = and(p,q)

nand( nand(p,p), nand(q,q) ) = or (p,q)
nor( nor(p,q) )              = or (p,q)
nor( nor(p), not(q) )        = or (p,q)
```

Gramática para Sintaxe de Expressões

- Símbolos não-terminais:

```
<Exp>, <Termo>, <Fator>
```

- Símbolos terminais (constantes):

```
~, |, &, (, ), a, b, ..., x, y, z, ...
```

- Símbolo inicial:

```
<Exp>
```

- Regras de produção/reconhecimento (EBNF):

```
<Exp>   ::= <Exp>   | <Termo>
          | <Termo>

<Termo> ::= <Termo> & <Fator>
          | <Fator>

<Fator> ::= a | b | ... | x | y | z | ...
          | ( <Exp> )
          | ~ <Fator>
```

- Reconhecimento de expressão válida sintaticamente

```
^                      <Exp>
|                        |
|             _____   <Termo>  _____
|            /           |          \
|           /            |           \
|        <Termo>         |         <Fator>
|           |            |            |
|    __  <Fator>  __     |            |
|   /       |        \   |            |
|  /      <Exp>       \  |            |
| |      /  |  \       | |            |
| |  <Exp>  |  <Termo> | |            |
| |    |    |     |    | |            |
| | <Termo> |  <Termo> | |            |
| |    |    |     |    | |            |
| | <Fator> |  <Fator> | |            |
| |    |    |     |    | |            |
|
| (    a    |     b    ) &            c
```

- Avaliação de expressão válida sintaticamente

```
| (a | b) & c
|   \ /     |
|    R1     |
|     \    /
|      \  /
v       R2
```

Simplificações de Expressões Lógicas e Circuitos

Expressões lógicas e seus circuitos equivalentes podem ser, muitas vezes, simplificados. Entretanto, nem todas as expressões podem ser minimizadas e, caso o forem, nem sempre haverá uma única expressão mínima global.

Dentre os métodos para simplificações destacam-se:

- As aplicações de regras da álgebra de *Boole*, principalmente, as que envolvem tautologias, contradições, negações, absorções e distribuições.
- Métodos Tabulares:
  1. Baseados em uma diferença (distância de *Hamming* = 1) ou $x.\ \ x.$:
     - Mapa de *Veitch-Karnaugh* (poucas varíáveis);
     - *Quine-McCluskey* (muitas variáveis).
  2. Baseados em diferenças (distância de *Hamming* = 2) ou $x.\ \ x.$ ou $x.\ \ x.$:
     - *Reed-Müller*.
  3. Inspeção visual da tabela e interpretação algorítmica :
     - Métodos envolvendo representações por grafos (BDD's) ou redes de conexões.

Distância *Hamming*

```
1100
1101
^
|__ 1 bit diferente -> dH = 1

1100
1110
^
|___ 1 bit diferente -> dH = 1

1101
1110
^^
||__ 2 bits diferentes -> dH = 2

1100
1001
^ ^
|_|__ 2 bits diferentes -> dH = 2

"casa"
```

```
"caso"
^
|_ 1 byte diferente -> dHc = 1
```

Código de Gray

- O código de Gray serve para expressar sequências binárias nas quais dois valores sucessivos tenham apenas um bit de diferença (distância de *Hamming* = 1).
- Código de Gray

```
0000
    \ dH = 1
    /
0001
    \ dH = 2
    /
0010
    \ dH = 1
    /
0011
    \ dH = 3
    /
0100
```

- Para 2 bits:

```
00  <----------+
    \ dH = 1   |
    /          |
01             |
    \ dH = 1   dH = 1
    /          |
11             |
    \ dH = 1   |
    /          |
10  <----------+
```

- Esse código também é conhecido com “código binário refletido“, por causa da característica abaixo:
- Gerador de código de Gray por espelhamento:

```
0    0     00  <- para 2 bits          0000  <- para 4 bits
1    1     01                          0001
    ---   ----                         0011
     1     11                          0010
     0     10                          0110
```

```
                                        0111
    00    000  <- para 3 bits           0101
    01    001                           0100
    11    011                          ------
    10    010                           1100
   ----  -----                          1101
    10    110                           1111
    11    111                           1110
    01    101                           1010
    00    100                           1011
                                        1001
                                        1000
```

Mapa de *Veitch-Karnaugh*

- Objetivo: Identificar a menor quantidade de grupos capaz de reunir todos os elementos.
- Para 2 variáveis:

```
                    0__
a\b    0      1     |
0    (0,0)  (0,1)
1    (1,0)  (1,1)
```

- Para 3 variáveis:

```
a\bc   00     01     11     10          a\bc  00  01  11  10
0    (0,00) (0,01) (0,11) (0,10)        0      0   1   3   2
1    (1,00) (1,01) (1,11) (1,10)        1      4   5   7   6


ab\c    0       1                       ab\c   0   1
00   (00,0)  (00,1)                     00     0   1
01   (01,0)  (01,1)                     01     2   3
11   (11,0)  (11,1)                     11     6   7
10   (10,0)  (10,1)                     10     4   5
```

- Para 4 variáveis:

  ab\cd 00 01 11 10 ab\cd 00 01 11 10
  00 (00,00) (00,01) (00,11) (00,10) 00 0 1 3 2
  01 (01,00) (01,01) (01,11) (01,10) 01 4 5 7 6
  11 (11,00) (11,01) (11,11) (11,10) 11 12 13 15 14
  10 (10,00) (10,01) (10,11) (10,10) 10 8 9 11 10

Importante

A ordem de leitura será da esquerda para a direita, da variável de ordem mais significativa para a de ordem menos significativa (linhas e colunas), semelhante à orientação usada para matrizes.

⚠ **Warning**

Se houver simplificações possíveis pela álgebra de *Boole*, o mapa de *Veitch-Karnaugh* não permitirá aplicá-las, caso a distância de *Hamming* for maior que 1 1.

Método de **Simplificação**

1. Por **Mintermos** :

```
Exemplo:
            0        1       2        6
SoP (0,1,2,6) = a'b'c' + a'b'c + a'b c' + a b c'

Por mintermos:

a\bc  00     01     11     10
0      1      1             1
1                            1

Grupos:
               0        1
par (m0,m1): a'b'c' + a'b'c  = (a'b').(c'+c) = (a'b').1 = a'b'
               2        6
par (m2,m6): a'b c' + a b c' = (b c').(a'+a) = (b c').1 = b c'
               0        2
par (m0,m2): a'b'c' + a'b c' = (a'c').(b'+b) = (a'c').1 = a'c'

grupos (pares) de mintermos: a'b' + b c' + a'c'
expressão simplificada por mintermos: a'b' + b c'

Nota: É redundante o termo: a'c' e pode ficar fora da expressão
simplificada.
      Pode ser removida por consenso:

      a'b' + b c' + a'c' -> (p.q)+(p'.r)+(q.r) = (p.q)+(p'.r)
      q.p  + p'.r + q.r  =  a'b' + b c'

Tabela-verdade
               DNF         CNF       Redundância
```

```
  abc  s     mintermos  MAXTERMOS
0 000  1  <- a'b'0                  <- a'.c'
1 001  1  <- a'b'1
2 010  1  <- 0 b c'                 <- a'.c'
3 011                 0 <- A +B'+C'
4 100                 0 <- A'+B +C
5 101                 0 <- A'+B +C'
6 110  1  <- 1 b c'
7 111                 0 <- A'+B'+C'

DNF - Disjunctive Normal Form (SoP)
CNF - Conjunctive Normal Form (PoS)
```

2. Por **MAXTERMOS** :

```
Exemplo:

A\BC    00       01      11       10
0                         0
1        0        0       0

              3          4          5          7
PoS (3,4,5,7) = (A +B'+C').(A'+B +C ).(A'+B +C').(A'+B'+C')

Grupos:
                  4          5
par (M4,M5): (A'+B +C ).(A'+B +C') = (A'+B )
                  3          7
par (M3,M7): (A +B'+C').(A'+B'+C') = (B'+C')
                  5          7
par (M5,M7): (A'+B +C').(A'+B'+C') = (A'+C')

grupos (pares) de MAXTERMOS: (A'+B ).(B'+C').(A'+C')

pelas propriedades da álgebra de Boole:

A'+B         . <- distributiva
B'+C'
--------------
A'B'+(B'B)     <- contradição
A'C'     +B C'
--------------
(A'B'+  0)      <- identidade
A'C'     +B C'
--------------
A'B'+
A'C'     +B C'
```

```
--------------
A'B'+A'C'+B C' <- comutativa

A'B'+B C'+A'C' <- expressão equivalente à dos mintermos

A'B'+A'C'+B C'
A'+C'          => testar se há redundância
--------------------
A'A'B'+A'A'C'+A'B C'
A'B'C'+A'C'C'+B C'C'
--------------------
A'B'  + A'C' +A'B C'
A'B'C'+ A'C' +B C'
--------------------
A'B'  + A'C' +A'B C'
A'B'C'       +  B C'
----------------------
A'B'(1+C')+ A'C'
          + B C'(A'+1)
----------------------
A'B'+ A'C'+ B C'

A'B'+ B C'+A'C' <- expressão equivalente à dos mintermos

expressão simplificada por MAXTERMOS: (A'+B ).(B'+C')

Nota: É redundante o termo: (A'+C') e pode ficar fora da expressão
simplificada.
      Pode ser removida por consenso.

Tabela-verdade
  ABC  S            Redund◆ncia
0 000
1 001
2 010
3 011  0 <- 0+B'+C'
4 100  0 <- A'+B+0
5 101  0 <- A'+B+1  <- A'+0+C'
6 110
7 111  0 <- 1+B'+C' <- A'+1+C
```

Método de Quine-McCluskey

- Exemplo para **mintermos** :

1. Agrupar valores segundo a quantidade de bits iguais a 1 (para facilitar a identificação de grupos com 1) :

```
- Grupo com 0 bits = 1 -> {0}

00  0000 a'b'c'd'  [x]

- Grupo com 1 bit  = 1 -> {1,2,8}

01  0001 a'b'c'd   [x][x]
02  0010 a'b'c d'  [x][x]
08  1000 a b'c'd'  [x][x]

- Grupo com 2 bits = 1 -> {5,6,10,12}

05  0101 a'b c'd      [x][x]
06  0110 a'b c d'     [x]
10  1010 a b'c d'     [x]
12  1100 a b c'd'     [x][x]

- Grupo com 3 bits = 1 -> {13}

13  1101 a b c'd         [x][x]

- Grupo com 4 bits = 1 -> {15}

15  1111 a b c d            [x]
```

2. Formar pares com 1 (testar valores de grupos vizinhos) :

```
- Com valores nos grupos 0 e 1:

par ( 0, 1) = (0000, 0001) = 000_       Ga
par ( 0, 2) = (0000, 0010) = 00_0  [x]
par ( 0, 8) = (0000, 1000) = _000  [x]

- Com valores nos grupos 1 e 2:

par ( 1, 5) = (0001, 0101) = 0_01       Gb
par ( 2, 6) = (0010, 0110) = 0_10       Gc
par ( 2,10) = (0010, 1010) = _010  [x]
par ( 8,10) = (1000, 1010) = 10_0  [x]
par ( 8,12) = (1000, 1100) = 1_00       Gd

- Com valores nos grupos 2 e 3:

par ( 5,13) = (1101, 0101) = _101       Ge
```

```
par (12,13) = (1100, 1101) = 110_       Gf

- Com valores nos grupos 3 e 4:

par (13,15) = (1101, 1111) = 11_1       Gg
```

3. Formar pares de pares com 1 :

```
par (( 0, 2), ( 8,10)) = ( 00_0, 10_0 )= _0_0   Gh
par (( 0, 8), ( 2,10)) = ( _000, _010 )= _0_0  [x]

Nota: Descartar o que não estiver em ordem.
      Parar, se não der para formar mais pares.
```

4. Identificar os grupos remanescentes -> Gx.
5. Montar tabela com os grupos identificados :

```
Gx  0  1  2  5  6  8 10 12 13 15
Ga  x  x                         000_
Gb     x     x                   010_
Gc        x     x                0_10
Gd                 x     x       1_00
Ge           x              x    _101
Gf                       x  x    110_
Gg                          x  x 11_1
Gh  x     x        x  x          _0_0
---------------------------------
    2  2  2  2  1  2  1  2  3  1
```

6. Identificar os grupos com apenas 1 ocorrência :

```
Gx  0  1  2  5  6  8 10 12 13 15
Ga  x  x                         000_
Gb     x     x                   010_
Gc        x     x                0_10
Gd                 x     x       1_00
Ge           x              x    _101
Gf                       x  x    110_
Gg                          x  x 11_1
Gh  x     x        x  x          _0_0
---------------------------------
    2  2  2  2  1  2  1  2  3  1
                  ^     ^        ^


- Marcar termos restantes não essenciais ('o') :

Gx  0  1  2  5  6  8 10 12 13 15
Ga  o  x
```

```
Gb  |  x     x
Gc  |     o    [x]                <= essenciais
Gd  |     |        o     x
Ge  |     |  x     |        o
Gf  |     |        |     x  o
Gg  |     |        |        x [x] <= essenciais
Gh  x     o        x [x]          <= essenciais
---------------------------------
    2  2  2  2  1  2  1  2  3  1
                ^     ^        ^


SoP = Gc + Gg + Gh + ...
```

7. Filtrar os essenciais :

```
Gx  0  1  2  5  6  8 10 12 13 15
Ga  o  x
Gb     x     x
Gd                 o     x
Ge           x              o
Gf                       x  o
---------------------------------
    1  2  0  2  0  1  0  2  2  0

- Descartar grupos com termos não-essenciais :

Gx  0  1  2  5  6  8 10 12 13 15
Ga  o  o                          <- descartar
Gb     x     x                    <- selecionar
Gd           |     o     x
Ge           o              o     <- descartar
Gf                       x  o
---------------------------------
    1  1  0  1  0  1  0  2  2  0

SoP = (Gc + Gg + Gh) + Gb + ...
```

8. Filtrar os selecionados :

```
Gx  0  1  2  5  6  8 10 12 13 15
Gd                 o     x
Gf                       x  o
---------------------------------
    0  0  0  0  0  0  0  2  0  0

Gx  0  1  2  5  6  8 10 12 13 15
Gd                 o     x        <- escolher arbitrariamente
```

```
Gf                        o  o     <- descartar
---------------------------------
    0  0  0  0  0  0  0  1  0  0

SoP = ((Gc + Gg + Gh) + Gb) + Gd

Nota:
    Para melhores escolhas, estabelecer critérios para filtrar.
```

9. Síntese :

```
SoP = (Gc  +  Gg  +  Gh)  +  Gb   +  Gd
    = 0_10 + 11_1 + _0_0  + 0_01  + 1_00
    = a'cd'+ abd  +  b'd' + a'c'd + ac'd'

      a'cd'+ abd  +  b'd' + a'c'd + ac'd'    (1)

- Pela álgebra de Boole ainda poderiam haver mais simplificações, mas o
método irá parar na expressão acima

a'cd' + a'c'd  = a'.(c d'+c'd ) = a'.(c^d)  -> dH = 2
a'cd' + a c'd' = d'.(a'c +a c') = d'.(a^c)  -> dH = 2
```

Método de Reed-Müller

- (2 -> XOR, XNOR).
- Circuitos combinacionais :

```
                                  s1    s0
n    m      M    a   b   a' b'  (a.b) (a^b)
0  a'.b'  A +B   0   0   1  1     0     0
1  a'.b   A +B'  0   1   1  0     0     1
2  a .b   A'+B   1   0   0  1     0     1
3  a .b   A'+B'  1   1   0  0     1     0

              0 + 0         =  0     0
              0 + 1         =  0     1
              1 + 0         =  0     1
              1 + 1         =  1     0

                                  s1    s0
n     m         M      a  b  c
0  a'.b'.c'  A +B +C   0  0  0    0     0
1  a'.b'.c   A +B +C'  0  0  1    0     1
2  a'.b .c'  A +B'+C   0  1  0    0     1
3  a'.b .c   A +B'+C'  0  1  1    1     0
```

```
4  a .b'.c'  A'+B +C   1  0  0    0     1
5  a .b'.c   A'+B +C'  1  0  1    1     0
6  a .b .c'  A'+B'+C   1  1  0    1     0
7  a .b .c   A'+B'+C'  1  1  1    1     1

                       3       5        6        7
s1 = SoP (3,5,6,7) = a'b c + a b'c  + a b c' + a b c

                       1       2        4        7
s0 = SoP (1,2,4,7) = a'b'c + a'b c' + a b'c' + a b c

Mapa VK :

s1
ab\c  0  1
00
01       1
11    1  1
10       1
```

- Simplificação/Minimização :

```
Ga = (3,7) = (011,111) = _11 = b c
Gb = (6,7) = (110,111) = 11_ = a b
Gc = (5,7) = (101,111) = 1_1 = a c

    3  5  6  7
Ga [x]       x
Gb       [x] x
Gc    [x]    x

s1 = Ga + Gb + Gc = b c + a b + a c

s0
ab\c  0  1
00       1
01    1
11       1
10    1

dH = 2 -> simplificação do tipo x.y+x.y' não se aplica
```

- Entretanto, pela álgebra de Boole :

```
s0 = SoP (1,2,4,7) = a'b'c + a'b c' + a b'c' + a b c

                   = a'(b'c+b c')   + a(b'c'+b c)
```

```
                    = a'(  b^c   )   + a(   b^c  )'
k = b^c
                    = a'    k        + a     k'

                    =             a^k
k = b^c
                    =             a^(b^c)

s0 = a ^ b ^ c

O mesmo se poderá encontrar usando os grupos nas diagonais:

(1,2) (001, 010) : (a'b'c +a'b c') = a'(b'c +b c') = a'(b^c)
(4,7) (100, 111) : (a b'c'+a b c ) = a (b'c'+b c ) = a (b^c)'

SoP(1,2,4,7) = a'(b^c) + a (b^c)'  = a^(b^c)
```

Representações Gráficas

```
    dH=1
0 ______ 1

    dH=1
00 ______ 01
 |         | dH=1
 |         |
10 ______ 11

       dH=1
  100 _______ 101
   /|         /| dH=1
000_|_____ 001 |
 |  |       |  |
 |110 ______|_111
 | /        | /  dH=1
010 ______ 011
```

Observação

Na **representação gráfica** se poderá verificar que os vértices (001 e 010) e (100 e 111) estão nas diagonais da face anterior e da posterior do cubo, respectivamente. Ao mesmo tempo, estão nas diagonais (001 e 100) e (010 e 111) da face superior e da inferior. Assim

também estão (001 e 111) e (010 e 100) nas faces à esquerda e à direita. Todas apresentam 2.